

UIU CSE Lecturer Demo + Q&A Cheatsheet

One-day cram manual for Demo Session B: self-introduction, 4-5 minute whiteboard class, and Q&A

Candidate	Ahmmad Nur Swapnil
Position	Lecturer, Department of CSE, United International University
Session	Demo Session B - June 20, 2026 - Reporting 11:00 AM
Scope	SPL, OOP, Discrete Math, DSA, Algorithms, DBMS, SE, OS, AI
How to use	Read pages 2-5 first. Then practice the demo scripts. Use the Q&A bank for rapid viva revision.

Priority rule: do not prepare like an exam. Prepare like a teacher who may be asked to explain any basic CSE concept clearly on a whiteboard.

Sources used: your UIU email, the provided UIU prep document, the UIU recruitment experience blog, public UIU career/mission pages, and recent publicly reported interview patterns.

0. Is the cheatsheet rich enough?

Yes - after adding the sections in this PDF. For one day, richness must mean coverage + prioritization + demo scripts, not encyclopedic length.

Need	Included here	Why it matters
Selection context	Process signals, expected behavior, likely panel priorities	You must know what the panel is evaluating.
Self-intro	30-60 sec script + Why UIU + Why hire + thesis answer	First impression should be controlled.
Demo format	5-minute board formula + board layout + teaching lines	Most candidates know content; fewer can teach it smoothly.
Subject prep	Nine-domain flash notes from SPL to AI	Random topic can come from any listed area.
Demo scripts	Ready-to-speak scripts for common topics	You need automatic flow under pressure.
Q&A bank	Likely questions + short answer style	The panel tests fundamentals and clarity.
Last-day plan	6-hour, 3-hour, 1-hour, and morning revision plan	One-day preparation needs triage.

Use this PDF in layers: first master the demo formula, then practice five high-probability topics, then revise the Q&A bank. Do not spend the day memorizing rare details.

1. What UIU is likely evaluating

Your email says the second phase assesses classroom presentation skills, clarity of communication, and subject knowledge. The provided UIU interview experience says the process commonly has CV screening, demo class, and final viva; the demo class usually uses basic SPL/OOP/DSA topics, has very short prep time in some years, and is followed by questions. Publicly reported interview patterns also mention OOP, DSA, algorithms, DBMS, OS, thesis, and basic programming questions.

Evaluation area	What the panel wants to see	How to show it
Classroom presence	Can you stand, write, explain, and control pace?	Write title, define, draw, explain, summarize.
Clarity	Can weak/average students understand you?	Use one tiny example; avoid jargon-first teaching.
Subject knowledge	Do you know the basic concept and common edge cases?	Mention complexity, limitation, or contrast.
Composure	Can you handle follow-up questions?	Pause, answer directly, then justify.
Academic fit	Do you sound like a faculty member?	Connect teaching, mentoring, and research.

2. The non-negotiable 4-5 minute demo formula

Time	Board/action	Speech objective
0:00-0:20	Write topic title	Signal structure and confidence.
0:20-0:50	One clean definition	Show you know the concept.
0:50-2:30	One example, diagram, or code	Turn abstract idea into visible object.
2:30-3:45	Step-by-step explanation	Teach, do not just list facts.
3:45-4:30	Use case / complexity / limitation	Show depth without overloading.
4:30-5:00	Summary in 2-3 bullets	Close cleanly and invite Q&A.

Universal board layout:

Topic: _____
Definition: _____
Example / Diagram / Code:
Step 1 -> Step 2 -> Step 3
Key points:
1. _____
2. _____
3. _____

Golden rule: every demo should contain one contrast. Example: array vs linked list, greedy vs DP, process vs thread, SQL vs NoSQL, supervised vs unsupervised.

3. Self-introduction and personal answers

30-60 second self-introduction:

Good morning respected faculty members. I am Ahmmad Nur Swapnil from BUET CSE. I recently defended my undergraduate thesis titled "LLM-Guided Security Analysis of Matter Protocol," where I worked on protocol security analysis using LLM-guided reasoning, formal reasoning, and practical validation.

Currently, I am serving as a Lecturer in the Department of CSE at Presidency University, where I teach core CSE theory and sessional courses, prepare lecture materials, conduct labs, and guide students in practical implementation.

My academic and professional interests include algorithms, data structures, machine learning, software systems, and security. I also have a competitive programming background, including Codeforces Expert rating and ICPC Dhaka Regional participation.

I am passionate about teaching fundamental CSE concepts in a simple and structured way, and I hope to contribute to UIU through effective teaching, student mentoring, and research-oriented academic work.

Question	Answer pattern
Why UIU?	UIU has a strong teaching and research culture. Its mission connects technical skill with practical and moral development. As a young faculty member, I want to grow where I can teach core CSE courses, mentor students, and gradually build research collaboration.
Why teaching?	I enjoy breaking difficult CSE concepts into simple examples. In my current lecturer role, I see that students understand better when theory, code, and real-world intuition are connected.
Why should we hire you?	I bring strong CSE fundamentals, recent teaching experience, research exposure, and competitive programming background. I can teach core CSE courses, conduct sessionals, guide students practically, and contribute to academic culture.
Favorite courses?	Pick three only: Data Structures, Algorithms, Object-Oriented Programming. These are safe because they match likely questions and your strengths.
Thesis in 40 sec?	My thesis analyzes the Matter IoT protocol using LLM-guided security reasoning and validation. The goal was to identify security-relevant protocol behavior and evaluate vulnerabilities or specification-level gaps through structured analysis.
If you do not know?	I am not fully certain about the exact answer at this moment. My understanding is _____. I would prefer to verify the precise detail rather than give an inaccurate answer.

4. Demo script bank - practice these aloud

SPL: Recursion

Definition: Recursion is a technique where a function solves a problem by calling itself on a smaller version of the same problem.

Board sketch / example:

```
int fact(int n) {
    if (n == 0) return 1; // base case
    return n * fact(n - 1); // recursive case
}

fact(4) = 4 * fact(3)
        = 4 * 3 * fact(2)
        = 4 * 3 * 2 * fact(1)
        = 24
```

Teaching flow	What to say
1	Start with the idea: solve a big problem using a smaller same problem.
2	Write two requirements: base case and recursive case.
3	Dry-run factorial with n = 4.
4	Mention stack memory and possible stack overflow.

Complexity / key property: Time depends on recurrence; factorial is $O(n)$ time and $O(n)$ call stack.

Likely question	Strong short answer
Recursion vs iteration?	Recursion is often cleaner for tree/divide-and-conquer problems; iteration often uses less memory.
What happens without base case?	Infinite recursion until stack overflow.
Is recursion always better?	No. It improves clarity in some problems but may cost stack memory.

Do not say: Do not say recursion is always slower or always better.

SPL: Pointer and call by reference

Definition: A pointer stores the memory address of another variable. It enables indirect access and modification.

Board sketch / example:

```
int x = 10;
int *p = &x;
*p = 20;
// now x becomes 20

void change(int *a) {
    *a = 50;
}
```

Teaching flow	What to say
1	Draw variable x as a box and p as another box storing x address.
2	Explain & gives address and * dereferences address.
3	Connect it to call by reference in C.
4	Mention arrays and dynamic memory briefly.

Complexity / key property: Pointer access is $O(1)$, but incorrect pointer use can crash the program.

Likely question	Strong short answer
Pointer vs normal variable?	Normal variable stores value; pointer stores address.
Why use pointer?	Modify original variable, dynamic memory, efficient data structures.
Danger?	Dangling pointer, null pointer dereference, memory leak.

Do not say: Do not confuse address-of operator & with logical AND &&.

OOP: Polymorphism and virtual function

Definition: Polymorphism means one interface can take many forms. In OOP it allows the same function call to behave differently for different objects.

Board sketch / example:

```
class Shape {
public:
    virtual void draw() { cout << "Shape"; }
};
class Circle : public Shape {
public:
    void draw() override { cout << "Circle"; }
};
Shape *s = new Circle();
s->draw(); // Circle
```

Teaching flow	What to say
1	Define polymorphism as one interface, many forms.
2	Show base class pointer pointing to child object.
3	Explain virtual enables runtime/dynamic binding.
4	Contrast overloading and overriding.

Complexity / key property: Runtime polymorphism uses dynamic dispatch; slight overhead, but flexible design.

Likely question	Strong short answer
Overloading vs overriding?	Overloading: same name different parameters, compile-time. Overriding: same signature in child class, runtime with virtual.
Why virtual?	To call child implementation through base pointer/reference.
Without virtual?	The base-class method may be called due to static binding.

Do not say: Do not say overriding needs different parameters. That is overloading.

OOP: Encapsulation

Definition: Encapsulation binds data and methods together and protects internal data using access modifiers.

Board sketch / example:

```
class BankAccount {
private:
    double balance;
public:
    void deposit(double x) { if (x > 0) balance += x; }
    double getBalance() { return balance; }
};
```

Teaching flow	What to say
1	State the security/control intuition.
2	Show private balance cannot be changed directly.
3	Show public methods validate access.
4	Contrast with abstraction.

Complexity / key property: Improves maintainability and prevents invalid states.

Likely question	Strong short answer
Encapsulation vs abstraction?	Encapsulation hides data; abstraction hides implementation complexity and exposes essential behavior.
Why private?	To prevent invalid direct modification.
Is getter/setter always good?	Not always; expose behavior, not unnecessary internals.

Do not say: Do not define encapsulation only as data hiding; mention binding data and methods too.

Discrete Math: Implication truth table

Definition: An implication $p \rightarrow q$ means if p is true, then q must be true. It is false only when p is true and q is false.

Board sketch / example:

```
p q p -> q
T T T
T F F
F T T
F F T
```

Example: If it rains, the road is wet.

Teaching flow	What to say
1	Define proposition and implication.
2	Draw the truth table.
3	Explain the only false case.
4	Mention contrapositive: $p \rightarrow q$ is equivalent to $\neg q \rightarrow \neg p$.

Complexity / key property: Used in proofs, program correctness, and conditions.

Likely question	Strong short answer
Why true when p is false?	Because the promise is not violated when the condition does not happen.
Converse?	$q \rightarrow p$, not generally equivalent.
Contrapositive?	$\neg q \rightarrow \neg p$, logically equivalent.

Do not say: Do not say implication means causation. It is a logical relation.

Discrete Math: Mathematical induction

Definition: Mathematical induction proves a statement for all natural numbers by proving a base case and an induction step.

Board sketch / example:

Claim: $1 + 2 + \dots + n = n(n+1)/2$

Base: $n = 1 \rightarrow 1 = 1(2)/2$

Assume true for $n = k$.

Show for $k+1$:

$1 + \dots + k + (k+1)$

$= k(k+1)/2 + (k+1)$

$= (k+1)(k+2)/2$

Teaching flow	What to say
1	Explain domino intuition.
2	Write base case.
3	Write induction hypothesis.
4	Use it to prove $k+1$.

Complexity / key property: Proof method, not algorithm complexity.

Likely question	Strong short answer
What are two parts of induction?	Base case and induction step.
Why need base case?	Without first domino, chain never starts.
Strong induction?	Assume all previous cases up to k , useful for recurrence/divisibility.

Do not say: Do not forget to explicitly state the induction hypothesis.

DSA: Middle of linked list

Definition: A linked list stores data in nodes, where each node contains a value and a pointer to the next node.

Board sketch / example:

```
Node* middle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

1 -> 2 -> 3 -> 4 -> 5
slow moves 1 step, fast moves 2 steps. Middle = 3

Teaching flow	What to say
1	Define linked list.
2	Explain slow-fast pointer intuition.
3	Dry-run with 5 nodes.
4	Mention even-node behavior.

Complexity / key property: $O(n)$ time, $O(1)$ extra space.

Likely question	Strong short answer
Why not count length?	Counting first needs two passes; slow-fast works in one pass.
Even number nodes?	This implementation returns the second middle.
Complexity?	$O(n)$ time and $O(1)$ space.

Do not say: Do not say linked list has $O(1)$ random access.

DSA: Hash table average $O(1)$ search

Definition: A hash table stores key-value pairs using a hash function that maps each key to an index.

Board sketch / example:

```
key = "cat"
hash(key) = 5
table[5] -> value
```

Collision:
table[5] -> (cat, 10) -> (act, 20)

Teaching flow	What to say
1	Define hash function.
2	Show key mapped to array index.
3	Explain collision and chaining.
4	Mention average vs worst-case.

Complexity / key property: Average $O(1)$, worst $O(n)$; resizing keeps performance stable.

Likely question	Strong short answer
Is search always $O(1)$?	Average-case $O(1)$, worst-case $O(n)$ if many collisions.
Collision solution?	Chaining or open addressing.
What is load factor?	Number of items divided by table size. High load factor increases collision.

Do not say: Do not claim hashing gives guaranteed $O(1)$ in all cases.

Algorithms: Greedy vs Dynamic Programming

Definition: Greedy makes the locally best choice. Dynamic programming solves overlapping subproblems and combines stored optimal sub-results.

Board sketch / example:

Fractional knapsack -> Greedy works.
0/1 knapsack -> Greedy can fail, DP needed.

DP idea for Fibonacci:
 $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$
Store answers to avoid recomputation.

Teaching flow	What to say
1	Define greedy in one sentence.
2	Define DP in one sentence.
3	Use knapsack contrast.
4	Mention optimal substructure and overlapping subproblems.

Complexity / key property: Greedy often $O(n \log n)$ or less; DP depends on state count.

Likely question	Strong short answer
Why use DP?	To avoid recomputing overlapping subproblems and ensure global optimum when recurrence is valid.
When greedy works?	When greedy-choice property holds.
DP drawback?	Extra memory and sometimes more implementation complexity.

Do not say: Do not say DP is always better than greedy. If greedy is valid, it is usually simpler and faster.

Algorithms: Dijkstra and negative edges

Definition: Dijkstra finds single-source shortest paths in graphs with non-negative edge weights.

Board sketch / example:

A --2--> B
A --5--> C
C --(-10)--> B

Dijkstra may finalize B too early before discovering shorter path through C.

Teaching flow	What to say
1	State assumption: all edge weights non-negative.
2	Explain finalized distance idea.
3	Show why negative edge breaks it.
4	Recommend Bellman-Ford for negative edges.

Complexity / key property: Non-negative weights only; use Bellman-Ford for negative edges.

Likely question	Strong short answer
Can Dijkstra work sometimes with negative edge?	It may accidentally work in some graphs, but it is not guaranteed.
Which algorithm handles negative edges?	Bellman-Ford, and it detects negative cycles.
Complexity?	Dijkstra with PQ: $O((V+E) \log V)$; Bellman-Ford: $O(VE)$.

Do not say: Do not say Dijkstra never works with negative edges; say not guaranteed/correct in general.

DBMS: ACID transactions

Definition: ACID properties ensure reliable execution of database transactions.

Board sketch / example:

Money transfer:
1. Deduct 100 from A
2. Add 100 to B

Atomicity: both happen or none.
Consistency: constraints remain valid.
Isolation: concurrent transfers do not interfere.
Durability: committed result survives crash.

Teaching flow	What to say
1	Define transaction.
2	Use bank transfer example.
3	Explain A, C, I, D one by one.
4	Mention why it matters in real systems.

Complexity / key property: Core DBMS reliability principle.

Likely question	Strong short answer
Atomicity vs durability?	Atomicity is all-or-nothing execution; durability is persistence after commit.
Isolation problem?	Dirty read, lost update, non-repeatable read.
Why transaction?	To keep data correct under failure and concurrency.

Do not say: Do not define consistency vaguely as same data; connect it to valid constraints/state.

DBMS: Normalization

Definition: Normalization organizes relational tables to reduce redundancy and update anomalies.

Board sketch / example:

Bad table:
StudentID, StudentName, Course1, Course2, DeptName

Better:
Student(StudentID, Name, DeptID)
Department(DeptID, DeptName)
Enrollment(StudentID, CourseID)

Teaching flow	What to say
1	Start with redundancy problem.
2	Show one bad table.
3	Split into meaningful tables.
4	Mention 1NF, 2NF, 3NF briefly.

Complexity / key property: Improves integrity; sometimes denormalization used for performance.

Likely question	Strong short answer
1NF?	Atomic values and no repeating groups.
2NF?	1NF plus no partial dependency on part of composite key.
3NF?	2NF plus no transitive dependency.

Do not say: Do not make normalization sound like always making more tables; it is controlled decomposition.

Software Engineering: SDLC and Agile

Definition: The Software Development Life Cycle describes phases used to build and maintain software systematically.

Board sketch / example:

Requirement -> Design -> Implementation -> Testing -> Deployment -> Maintenance

Waterfall: sequential
Agile: iterative and adaptive

Teaching flow	What to say
1	Draw SDLC pipeline.
2	Explain why requirement comes first.
3	Contrast Waterfall and Agile.
4	Mention testing/maintenance as continuous concerns.

Complexity / key property: Choose model based on project uncertainty and stakeholder needs.

Likely question	Strong short answer
Functional vs non-functional?	Functional says what system does; non-functional says quality constraints like performance/security.
Agile advantage?	Frequent feedback and adaptability.
Waterfall advantage?	Clear documentation and predictable phases for stable requirements.

Do not say: Do not say Agile means no planning or no documentation.

Software Engineering: SOLID principles

Definition: SOLID is a set of object-oriented design principles that improves maintainability and extensibility.

Board sketch / example:

S - Single Responsibility
O - Open/Closed
L - Liskov Substitution
I - Interface Segregation
D - Dependency Inversion

Teaching flow	What to say
1	Write SOLID expansion.
2	Explain S and O with tiny example.
3	Mention that these are design guidelines, not syntax.
4	Connect to maintainable code.

Complexity / key property: Useful for large-scale maintainable OO software.

Likely question	Strong short answer
Most important for beginners?	Single Responsibility: one class should have one main reason to change.
Open/Closed?	Open for extension, closed for modification.
Dependency inversion?	Depend on abstraction, not concrete implementation.

Do not say: Do not over-explain all five in a demo unless asked.

Operating Systems: Process vs Thread

Definition: A process is a program in execution; a thread is a lightweight execution unit inside a process.

Board sketch / example:

Process P1: code, data, heap, stack
Thread T1: own stack
Thread T2: own stack
Threads share code/data/heap of the process.

Teaching flow	What to say
1	Define process.
2	Define thread.
3	Draw shared memory vs separate memory.
4	Mention context switching and communication.

Complexity / key property: Threads improve concurrency but need synchronization.

Likely question	Strong short answer
Why thread is lightweight?	Threads share process address space, so creation/switching is usually cheaper.
Can one thread crash process?	Yes, since threads share memory.
Process communication?	Uses IPC such as pipe, message queue, shared memory, socket.

Do not say: Do not say threads have no separate memory; each has its own stack/register context.

Operating Systems: Deadlock

Definition: Deadlock occurs when processes wait forever because each holds a resource and waits for another resource held by another process.

Board sketch / example:

P1 holds R1 and waits for R2
P2 holds R2 and waits for R1

Deadlock conditions:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Teaching flow	What to say
1	Define deadlock.
2	Draw two-process cycle.
3	List four Coffman conditions.
4	Mention prevention by breaking at least one condition.

Complexity / key property: OS synchronization and resource-allocation issue.

Likely question	Strong short answer
How prevent deadlock?	Break one of the four necessary conditions.
Deadlock vs starvation?	Deadlock: waiting cycle; starvation: a process waits indefinitely due to scheduling/resource policy.
Detection?	Resource allocation graph or wait-for graph.

Do not say: Do not confuse deadlock with simple blocking.

AI: AI vs ML vs DL

Definition: Artificial Intelligence is the broad field of making machines act intelligently; Machine Learning is a subset where systems learn from data; Deep Learning is ML using deep neural networks.

Board sketch / example:

AI
ML
DL

Example:

AI: rule-based chess agent

ML: spam classifier

DL: CNN image classifier

Teaching flow	What to say
1	Draw nested circles.
2	Define each level.
3	Give one example each.
4	Mention data dependence increases from AI to ML/DL.

Complexity / key property: Good opening topic if AI is selected.

Likely question	Strong short answer
Supervised learning?	Learns from labeled input-output examples.
Unsupervised learning?	Finds structure without labels, such as clustering.
Overfitting?	Model performs well on training data but poorly on unseen data.

Do not say: Do not say all AI is ML. Rule-based AI exists.

AI: Confusion matrix and evaluation

Definition: A confusion matrix summarizes classification outcomes using true positive, false positive, true negative, and false negative.

Board sketch / example:

Predicted + Predicted -
Actual + TP FN
Actual - FP TN

Accuracy = $(TP+TN)/(TP+FP+FN+TN)$

Precision = $TP/(TP+FP)$

Recall = $TP/(TP+FN)$

Teaching flow	What to say
1	Define classification evaluation.
2	Draw 2x2 matrix.
3	Explain TP, FP, FN using disease/spam example.
4	Mention precision-recall tradeoff.

Complexity / key property: Basic ML evaluation concept.

Likely question	Strong short answer
Precision vs recall?	Precision: how many predicted positives are correct. Recall: how many actual positives are found.
When recall important?	Medical screening, fraud detection, safety-critical alerts.
F1 score?	Harmonic mean of precision and recall.

Do not say: Do not use accuracy alone for imbalanced data.

5. Rapid revision tables by domain

Structured Programming Language - must know

Topic	One-line answer	Common follow-up
Data type	Defines type and memory interpretation of value: int, float, double, char.	Signed vs unsigned range.
Array	Contiguous collection of same-type elements accessed by index.	Random access $O(1)$, insertion middle $O(n)$.
Structure	User-defined type grouping different fields under one name.	struct vs union: union shares memory.
Pointer	Variable storing address of another variable.	Null/dangling pointer, & and * operators.
Function	Reusable block of code with parameters and return type.	Call by value vs reference.
Recursion	Function calls itself on smaller input.	Base case, recursive case, stack overflow.
Loop	Repeats block while condition/iteration holds.	for known count, while condition-based, do-while runs at least once.
Dynamic memory	Memory allocated at runtime using malloc/free or new/delete.	Memory leak if not freed.

OOP - must know

Concept	Simple definition	Trap to avoid
Class	Blueprint for objects.	A class itself may not allocate object memory until object is created.
Object	Runtime instance of a class.	Object has state and behavior.
Encapsulation	Bind data and methods, protect data with access modifiers.	Not only hiding; also controlled access.
Abstraction	Expose essential interface, hide implementation details.	Different from encapsulation.
Inheritance	Child class reuses/extends parent class.	Java supports single class inheritance, multiple interfaces.
Polymorphism	Same interface, different behavior.	Overloading != overriding.
Constructor	Special method to initialize object.	No return type.
Destructor	Called when object is destroyed in C++.	Java has garbage collection, not deterministic destructor.
Friend function	Non-member function allowed to access private/protected members in C++.	Friend is not inherited; not class member.

Data Structures - operation complexity

Structure	Access/Search	Insert/Delete	Best demo angle
Array	Access $O(1)$, search $O(n)$	Middle insert/delete $O(n)$	Contiguous memory and index formula.
Linked list	Access/search $O(n)$	$O(1)$ if node pointer known; otherwise search needed	Node + pointer, dynamic memory.
Stack	Top access $O(1)$	push/pop $O(1)$	LIFO: undo, recursion stack.
Queue	Front access $O(1)$	enqueue/dequeue $O(1)$	FIFO: scheduling, BFS.
BST	Average $O(\log n)$, worst $O(n)$	Average $O(\log n)$, worst $O(n)$	Left smaller, right larger.
Heap	Find min/max $O(1)$	insert/delete $O(\log n)$	Priority queue.
Hash table	Average $O(1)$, worst $O(n)$	Average $O(1)$	Hash function + collision.
Graph	Depends on representation	Adjacency list vs matrix	Vertices and edges; BFS/DFS.

Algorithms - high-yield facts

Topic	Core idea	Complexity / caveat
Binary search	Repeatedly halve sorted search range.	$O(\log n)$, array must be sorted.
Bubble sort	Adjacent swaps; largest bubbles to end.	$O(n^2)$, stable, best $O(n)$ with flag.
Insertion sort	Insert current element into sorted prefix.	$O(n^2)$, good for nearly sorted data.
Selection sort	Repeatedly select minimum.	$O(n^2)$, not stable by default.
Merge sort	Divide and conquer; merge sorted halves.	$O(n \log n)$, stable, extra memory.
Quick sort	Partition around pivot.	Average $O(n \log n)$, worst $O(n^2)$.
BFS	Level-order graph traversal using queue.	$O(V+E)$, shortest path in unweighted graph.
DFS	Go deep using stack/recursion.	$O(V+E)$, cycle/topological applications.

Topic	Core idea	Complexity / caveat
Dijkstra	Greedy shortest path.	Non-negative weights only.
Kruskal	MST by sorting edges and DSU.	$O(E \log E)$.
Prim	MST by growing from a node.	$O(E \log V)$ with PQ.
Topological sort	Linear order of DAG dependencies.	Only for directed acyclic graphs.

DBMS - high-yield facts

Concept	Answer	Example/extra
Primary key	Uniquely identifies each row.	StudentID.
Foreign key	References primary key of another table.	Enrollment.StudentID -> Student.StudentID.
Candidate key	Minimal attribute set that can identify row.	Email or StudentID.
Super key	Any attribute set that identifies row; may contain extra attributes.	{StudentID, Name}.
Join	Combines rows from tables using related columns.	INNER, LEFT, RIGHT, FULL.
Index	Data structure that speeds lookup.	Improves read, may slow write.
Transaction	Logical unit of work.	Bank transfer.
ACID	Atomicity, Consistency, Isolation, Durability.	Reliability guarantee.
Normalization	Reduce redundancy and anomalies.	1NF, 2NF, 3NF, BCNF.
SQL vs NoSQL	Relational/schema vs flexible/non-relational.	SQL vertical; NoSQL often horizontal.

Operating Systems - high-yield facts

Concept	Answer	Follow-up
Process	Program in execution with its own address space.	PCB stores state.
Thread	Lightweight execution unit inside process.	Shares process memory.
Context switch	CPU switches from one process/thread to another.	Has overhead.
Scheduling	Decides which process gets CPU.	FCFS, SJF, Round Robin, Priority.
Deadlock	Processes wait forever in circular dependency.	Four Coffman conditions.
Mutex	Lock with one owner for mutual exclusion.	Binary ownership.
Semaphore	Counter controlling access to resources.	Counting or binary.
Paging	Fixed-size virtual memory pages mapped to frames.	Avoids external fragmentation.
Virtual memory	Gives process illusion of large memory.	Uses disk as backing store.
Cache	Small fast memory close to CPU.	Reduces average memory access time.

Artificial Intelligence - high-yield facts

Concept	Answer	Example
AI	Broad field of intelligent machine behavior.	Rule-based game agent.
ML	Subset of AI that learns from data.	Spam classification.
DL	Subset of ML using deep neural networks.	CNN image classifier.
Supervised	Learn from labeled examples.	Disease yes/no.
Unsupervised	Find patterns without labels.	Customer clustering.
Reinforcement	Agent learns actions by rewards.	Game-playing agent.
Overfitting	Good train performance, poor test performance.	Too complex model.
Underfitting	Too simple model; poor train/test performance.	Linear model for nonlinear data.
Precision	$TP/(TP+FP)$.	Correctness among predicted positives.
Recall	$TP/(TP+FN)$.	Coverage among actual positives.

6. High-probability Q&A bank

SPL / C

Question	Answer
What is the difference between structure and union?	A structure allocates memory for all fields; a union shares one memory block among all fields, so only one value is valid at a time.
What is a dangling pointer?	A pointer that refers to memory that has already been freed or gone out of scope.
What is memory leak?	Allocated memory that is no longer reachable and not freed.
What is the difference between malloc and calloc?	malloc allocates uninitialized memory; calloc allocates and initializes memory to zero.
What is static variable?	A variable that retains its value between function calls or has class-level storage depending on context.
Call by value vs call by reference?	Value passes copy; reference/address passes access to original variable.
When use switch instead of if-else?	When checking one expression against multiple constant cases.
Why use functions?	Reusability, modularity, readability, testing.

OOP

Question	Answer
What are four pillars of OOP?	Encapsulation, abstraction, inheritance, polymorphism.
What is the diamond problem?	Ambiguity in multiple inheritance when two parent paths inherit from the same base class; C++ solves with virtual inheritance.
Does Java support multiple inheritance?	Java does not support multiple inheritance of classes, but supports multiple interfaces.
Constructor vs method?	Constructor initializes object and has no return type; method defines object behavior and can return values.
Private vs protected vs public?	Private within class, protected within class and derived classes, public from outside.
Abstract class vs interface?	Abstract class can contain state and implemented methods; interface defines behavior contract, though modern Java interfaces can have default methods.
Static binding vs dynamic binding?	Static binding resolved at compile time; dynamic binding resolved at runtime based on actual object type.
Why use inheritance?	Code reuse and modeling is-a relationship, but composition may be better when relationship is has-a.

Data Structures

Question	Answer
Array vs linked list?	Array has contiguous memory and $O(1)$ index access; linked list has dynamic nodes and easier insert/delete when node pointer is known.
Stack applications?	Function calls, recursion, undo, expression evaluation, DFS.
Queue applications?	Scheduling, printer queue, BFS, buffering.
BST worst case?	$O(n)$, when tree becomes skewed.
Heap vs BST?	Heap gives quick min/max, not ordered search; BST supports ordered search/traversal.
Graph representation?	Adjacency matrix uses $O(V^2)$, good for dense graph; adjacency list uses $O(V+E)$, good for sparse graph.
Circular queue?	Queue implementation using circular array to reuse freed spaces.
What is collision in hashing?	Two different keys map to same table index.

Algorithms

Question	Answer
BFS vs DFS?	BFS explores level by level using queue; DFS explores deep using stack/recursion.
Which gives shortest path in unweighted graph?	BFS.
Why quicksort worst $O(n^2)$?	Bad pivot choices create highly unbalanced partitions.
Stable sorting?	Equal elements keep original relative order.
Greedy-choice property?	A locally optimal choice can lead to a globally optimal solution.
Optimal substructure?	Optimal solution can be built from optimal subsolutions.
Overlapping subproblems?	Same subproblems repeat many times; memoization/tabulation helps.
Topological sort works on?	Directed acyclic graph only.
MST algorithms?	Kruskal and Prim.
Dijkstra negative edge?	Not guaranteed correct; use Bellman-Ford for negative edges.

DBMS

Question	Answer
Primary key vs unique key?	Primary key uniquely identifies row and cannot be NULL; unique key also enforces uniqueness but may allow NULL depending on DBMS.
Delete vs truncate?	DELETE removes selected rows and can have WHERE; TRUNCATE removes all rows faster and is DDL-like in many systems.
INNER JOIN vs LEFT JOIN?	INNER returns matching rows only; LEFT returns all left table rows and matching right rows if available.
Why index?	To speed search and join operations, with storage and write overhead.
What is anomaly?	Insertion, update, or deletion problem caused by poor table design.
What is transitive dependency?	Non-key attribute depends on another non-key attribute.
What is isolation level?	Rule controlling visibility of concurrent transaction effects.
SQL injection?	Attack where untrusted input changes SQL query meaning; prevent with parameterized queries.

Software Engineering

Question	Answer
Functional vs non-functional requirement?	Functional describes features; non-functional describes quality constraints such as security/performance.
Verification vs validation?	Verification: are we building the product right? Validation: are we building the right product?
Unit vs integration testing?	Unit tests one module; integration tests interactions among modules.
Agile vs Waterfall?	Waterfall is sequential; Agile is iterative with frequent feedback.
Cohesion vs coupling?	High cohesion inside module is good; low coupling between modules is good.
Design pattern?	Reusable solution template for common design problems.
Singleton pattern?	Ensures one instance and provides global access point.
Observer pattern?	Subject notifies dependent observers when state changes.

Operating Systems

Question	Answer
Mutex vs semaphore?	Mutex has ownership and mutual exclusion; semaphore is a signaling/counting mechanism.
Deadlock conditions?	Mutual exclusion, hold and wait, no preemption, circular wait.
Starvation vs deadlock?	Starvation is indefinite waiting due to unfair allocation; deadlock is circular waiting.
Paging vs segmentation?	Paging fixed-size blocks; segmentation logical variable-size blocks.
Page fault?	Required page is not in physical memory; OS loads it from disk.
Thrashing?	Too much paging causing CPU to spend most time swapping pages.
Round Robin?	CPU scheduling where each process gets fixed time quantum.
User mode vs kernel mode?	User mode has restricted privileges; kernel mode can execute privileged operations.

AI / ML

Question	Answer
Classification vs regression?	Classification predicts category; regression predicts continuous value.
Precision vs recall?	Precision measures correctness of positive predictions; recall measures coverage of actual positives.
F1 score?	Harmonic mean of precision and recall.
How reduce overfitting?	More data, regularization, simpler model, dropout, early stopping, cross-validation.
Training vs validation vs test?	Train fits model; validation tunes model; test estimates final generalization.
A* evaluation function?	$f(n) = g(n) + h(n)$, cost so far plus heuristic estimate.
Admissible heuristic?	Never overestimates true remaining cost.
Clustering example?	K-means grouping customers by behavior.

Discrete Math

Question	Answer
Proposition?	A declarative statement that is either true or false.
Predicate?	Statement with variables whose truth depends on variable values.
Universal quantifier?	For all; symbol $\forall$.

Question	Answer
Existential quantifier?	There exists; symbol exists.
Contrapositive?	$p \rightarrow q$ is equivalent to $\neg q \rightarrow \neg p$.
Converse?	$q \rightarrow p$; not generally equivalent to $p \rightarrow q$.
Function?	Relation where each input maps to exactly one output.
Equivalence relation?	Reflexive, symmetric, transitive relation.

7. Dangerous questions - answer exactly

Question	Bad answer	Safe answer
Can Dijkstra handle negative edges?	No, never.	Not guaranteed correct in general. It may work accidentally in some cases, but the assumption is non-negative edges. Use Bellman-Ford.
Is recursion better than iteration?	Yes, recursion is better.	Depends. Recursion is clearer for recursive structures; iteration often uses less memory.
Is hash search $O(1)$?	Always $O(1)$.	Average-case $O(1)$, worst-case $O(n)$ due to collision.
Is Java multiple inheritance supported?	Yes/No without detail.	Java does not support multiple inheritance of classes, but supports multiple interfaces.
Is encapsulation just data hiding?	Yes.	It binds data and methods together and controls access; data hiding is one effect.
Does SQL scale worse than NoSQL?	Yes always.	It depends. SQL is relational and often vertically scaled; NoSQL systems are often designed for horizontal scalability.
Does AI mean machine learning?	Yes.	ML is a subset of AI; AI also includes rule-based/search-based systems.
Does deadlock mean high waiting time?	Yes.	No. Deadlock specifically requires circular waiting where processes wait indefinitely.

8. Whiteboard practice prompts

Practice these exactly as 4-minute talks. Use a phone timer. Stop at 4:30 even if unfinished; in the actual session, controlled ending matters.

Priority	Topic	Must include
1	Polymorphism	virtual function, base pointer, overloading vs overriding
2	Middle of linked list	slow-fast pointer, $O(n)$, $O(1)$
3	DP vs Greedy	0/1 vs fractional knapsack, overlapping subproblems
4	ACID	bank transfer example
5	Process vs Thread	memory sharing, context switch, synchronization
6	Hash Table	hash function, collision, average/worst case
7	Dijkstra	non-negative weight assumption, Bellman-Ford
8	Normalization	bad table -> decomposed tables, 1NF/2NF/3NF
9	Implication truth table	only false case, contrapositive
10	AI vs ML vs DL	nested relation and examples

Micro-teaching rubric for yourself:

Score	Question
0/1	Did I define the topic in one clear sentence?
0/1	Did I draw or write something concrete on the board?
0/1	Did I explain step by step rather than just listing facts?
0/1	Did I mention complexity/use case/limitation?
0/1	Did I summarize in 2-3 points?

9. One-day preparation plan

Block	Time	Task	Output
Block 1	60 min	Self-intro + demo formula + favorite-course answers	You can start confidently.
Block 2	90 min	OOP + DSA	Polymorphism, linked list, hash, stack/queue ready.
Block 3	90 min	Algorithms + DBMS	DP/greedy, Dijkstra, sorting, ACID, normalization ready.
Block 4	60 min	OS + AI + Discrete	Process/thread, deadlock, AI/ML/DL, implication/induction ready.
Block 5	60 min	Practice five 4-minute demos aloud	Muscle memory for board teaching.
Block 6	45 min	Q&A bank rapid pass	Short answers loaded.
Block 7	15 min	Documents, dress, route, sleep	Avoid logistics failure.

If only 3 hours remain: practice Polymorphism, Middle Linked List, DP vs Greedy, ACID, Process vs Thread. Then read dangerous questions. If only 1 hour remains: self-intro, board formula, and the dangerous questions table.

10. Final-hour revision sheet

Concept	One sentence to remember
OOP	Encapsulation controls data; abstraction controls complexity; inheritance reuses; polymorphism changes behavior through same interface.
Linked list	Slow-fast pointer finds middle in one pass: slow moves 1, fast moves 2.
Hash table	Average $O(1)$, worst $O(n)$; collision handled by chaining/open addressing.
DP vs greedy	Greedy local choice; DP stores overlapping subproblem results for global optimum.
Dijkstra	Works for non-negative weights; negative edges need Bellman-Ford.
DBMS	ACID: all-or-nothing, valid state, independent concurrency, persistent commit.
OS	Process has own address space; threads share process memory but have own stack/registers.
Deadlock	Mutual exclusion + hold/wait + no preemption + circular wait.
AI	AI broad, ML learns from data, DL uses deep neural networks.
Discrete	$p \rightarrow q$ is false only when p true and q false.

Final behavior checklist:

Before entering	During demo	During Q&A
Phone silent; formal dress; marker ready; self-intro memorized.	Write title; define; example; step-by-step; complexity/use case; summary.	Listen fully; pause; answer directly; give example; admit uncertainty if needed.

11. Sources consulted

Source	How it was used
UIU email invitation from Department of CSE	Confirmed current session date, reporting time, format, domains, and evaluation criteria.
UIU Prep Google Doc: https://docs.google.com/document/d/1Y3JrYGUXGX7zMeh2xa2Ye-jCt78_851q9gaCOvDo2pc/mobilebasic	Topic inventory from previous prep guide: SPL, OOP, DSA, DBMS, sorting, graph algorithms, network basics, SOLID, design patterns.
Lecturer Interview Experience - UIU: https://asif-saad.github.io/blog/university-recruitment/UIU-recruitment/UIU-recruitment-experience/	Recruitment process pattern, English demo advice, sample demo/viva questions, favorite-course warning, and behavior tips.
UIU career pages: https://www.uiu.ac.bd/career/lecturer-cse/ and related pages	Public eligibility and presentation/interview signal.
UIU mission page / public UIU pages	Used for a careful Why UIU answer aligned with teaching, learning, research, and practical skill development.
Recent public interview reports such as Glassdoor	Cross-checked likely question patterns such as OOP, DSA, algorithm complexity, cache, Dijkstra, hash O(1), linked-list middle, and thesis questions.

Note: older guide contents may not exactly match this year. Your current email is the authority for the topic domains, so this PDF expands beyond older SPL/OOP/DSA-heavy lists.